Individual Project Report

Empirical Analysis of Scale, Tokenization, and Attention in Hindi and Nepali

Srinivas Billuri · *Roll no : 2023101029*

Final Consolidated Project Report · September 2026

1 Introduction & Research Objectives

The Autoregressive language models rely critically on the interaction between corpus scale, pre-processing quality, tokenizer vocabulary design, and architectural capacity. In low-resource settings, training small-scale language models (∼25M parameters) may cause unique challenges like tokenizers often suffer from excessive fertility (splitting words into many subwords), web-scraped data contains severe OCR and scraping noise, and downstream fine-tuning can rapidly induce catastrophic forgetting of foundational syntax.

This project investigates these dynamics across two Indic languages sharing the Devanagari script:

- **Hindi (Model H, Higher-Resource):** A widely spoken language with extensive digital resources, including Wikipedia, Wikisource, Wikibooks, Wikiquote, and large-scale filtered corpora (IndicCorpV2).
- **Nepali (Model L, Lower-Resource):** A lower-resource language requiring aggressive data harvesting, combining open web corpora, optical character recognition (OCR), custom news scrapers, and IndicCorpV2.

The project has three phases:

- **Phase 1 (Data & Tokenization):** Data collection, cleaning, MinHash LSH deduplication, and SentencePiece UNIGRAM tokenizer training.
- **Phase 2 (Pretraining & Evaluation):** Architecture design, 16,000-step pretraining on Kaggle T4 GPUs, perplexity/BPB evaluation, multi-temperature generation, and 64-head attention analysis.
- **Phase 3 (Finetuning & Analysis):** Synthetic comparative reasoning finetuning, Pareto sizing sweeps, pretrain vs. finetune attention analysis, error fingerprinting, and a No-RoPE positional encoding ablation.

2 Data Collection, Cleaning & Tokenization (Phase 1)

2.1 Corpus Provenance and Pre-processing Pipeline

The raw corpora were assembled from both curated community dumps and raw web extractions. For Hindi, the sources included Wikipedia (41.79 M raw tokens), Wikisource (51.44 M), Wiktionary, Wikibooks, Wikiquote (6.70 M), web news (0.19 M), and IndicCorpV2 (485.52 M). For Nepali, Wikipedia provided only 7.32 M raw tokens, necessitating additional sources including OCR extractions (0.35 M), custom scrapers from BBC Nepali and Setopati (48.36 M), and IndicCorpV2 (455.41 M).

Both corpora underwent a standardized 7-stage cleaning pipeline:

1. **Short Document Removal:** Documents which have fewer than 50 characters were removed.
2. **FastText Language Identification:** Filtered using `lid.176.bin`. Hindi used a strict threshold of ≥ 0.50; Nepali used a calibrated threshold of ≥ 0.30 to retain volume from noisier web and OCR sources.
3. **Unicode Normalization:** Applied canonical NFC normalization for all the documents.
4. **Devanagari Sanitization:** Stripped Zero-Width Joiners (ZWJ, `U+200D`), Zero-Width Non-Joiners (ZWNJ, `U+200C`), lingering HTML/XML markup, and non-Devanagari ASCII control artifacts.
5. **Exact Deduplication:** Full-document SHA-256 hash deduplication.
6. **Near-Deduplication:** MinHash LSH near-deduplication using 64 permutations and with a Jaccard similarity threshold of 0.80.
7. **Deterministic Splitting:** Partitioned determistically into fixed 96% Train, 2% Validation, and 2% Test sets.

Corpus Preprocessing Statistics			SentencePiece UNIGRAM Tokenizer		
Corpus Metric	**Hindi (Model H)**	**Nepali (Model L)**	**Tokenizer Metric**	**Hindi (8K)**	**Nepali (4K)**
Raw Documents	9.13 M	14.96 M	Algorithm	UNIGRAM	UNIGRAM
Cleaned Documents	7.75 M	11.96 M	Vocabulary Size (V)	**8,000**	**4,000**
Cleaning Yield	84.9%	80.0%	Token Fertility (tok/wd)	**1.380**	**1.698**
Final Word Tokens	585.6 M	511.4 M	Out-of-Vocab (UNK) Rate	0.1508%	0.1098%
Train Tokens (96%)	794.5 M	868.3 M	Avg Chars per Token	3.004	3.227
Val / Test Tokens (2%)	16.5M / 16.6M	18.1M / 18.0M	Character Coverage	95.93%	101.15%
Test UTF-8 Bytes	154.3 MB	161.6 MB	Byte Fallback Enabled	False	False

Table 1: Corpus preprocessing statistics (left) and SentencePiece UNIGRAM tokenizer parameters (right) for Hindi and Nepali.

2.2 Tokenizer Selection and The Fertility Paradox

Tokenizers were trained using SentencePiece UNIGRAM on 1 M sampled sentences with `byte_fallback=False`. Vocabulary sweeps evaluated UNK rates and parameter trade-offs:

- **Hindi Sweep ($\{8\mathbf{K}, 16\mathbf{K}, 24\mathbf{K}, 32\mathbf{K}, 48\mathbf{K}\}$):** 8K was selected for its low UNK rate (0.1508%) while allocating 84% of parameters to Transformer layers.
- **Nepali Sweep ($\{4\mathbf{K}, 8\mathbf{K}, 16\mathbf{K}, 24\mathbf{K}\}$):** 4K was selected for minimal UNK rate (0.1098%) and 101.15% character coverage with a compact 2.05M embedding table.

The Fertility Paradox: Although Hindi has more raw word tokens (585.6M vs. 511.4M), Nepali's higher fertility (1.698 vs. 1.380) generates *more* training subwords (868.3M vs. 794.5M). Because Nepali decomposes words into 23% more pieces, every concept requires more autoregressive generation steps, amplifying cumulative error probability.

3 Model Architecture & Pretraining Dynamics (Phase 2)

3.1 Architectural Design and Capacity Calibration

Both models are decoder-only GPT-style Transformers built completely from scratch in PyTorch, avoiding any black-box wrappers. Core architectural elements include: Rotary Position Embeddings (RoPE, base = 10,000), SwiGLU feed-forward networks ($\text{SwiGLU}(x) = (\text{Swish}(xW_{\text{gate}}) \odot xW_{\text{up}})W_{\text{down}}$), Pre-Layer Normalization, and tied input-output embeddings.

Capacity Calibration: Hindi's 8K vocabulary requires $V \times d_{\text{model}} = 8000 \times 512 = 4,096,000$ embedding weights, whereas Nepali's 4K vocabulary requires only $2,048,000$. Using identical feed-forward dimensions would have left Nepali under-parameterized by over 2M parameters. To enforce a fair, controlled comparison, Nepali's feed-forward intermediate dimension was calibrated upward to $d_{\text{ff}} = 1,184$ (vs. $1,024$ in Hindi). This equalized total capacity: **25.08M (Model H) vs. 25.00M (Model L)**, matching non-embedding parameter capacity within 0.3%.

Architectural Feature	Model H (Hindi)	Model L (Nepali)	No-RoPE Ablation
Total Trainable Parameters	**25,084,928**	**25,003,008**	**25,084,928**
Vocabulary Size (V)	8,000	4,000	8,000
Embedding Weights ($V \times d_{\text{model}}$)	4,096,000 (16.3%)	2,048,000 (8.2%)	4,096,000
FFN Intermediate (d_{ff})	**1,024**	**1,184**	1,024
SwiGLU FFN Parameters	12,582,912 (50.2%)	14,548,992 (58.2%)	12,582,912
Attention Parameters (8 layers)	8,388,608 (33.4%)	8,388,608 (33.5%)	8,388,608
Hidden Dim / Heads / Layers	512 / 8 / 8	512 / 8 / 8	512 / 8 / 8
Context Window / Dropout	512 / 0.1	512 / 0.1	512 / 0.1
Positional Encoding	RoPE (base = 10^4)	RoPE (base = 10^4)	**None**
Hardware / Training Time	1× T4 (4.7h)	1× T4 (4.8h)	1× T4 (4.7h)

Table 2: Architectural configuration and parameter calibration of the Hindi and Nepali models, including the No-RoPE ablation.

3.2 Pretraining Dynamics and Convergence

Pretraining was executed on a single Kaggle NVIDIA Tesla T4 (16GB) GPU for 16,000 steps using AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$, weight decay 0.1), peak learning rate 3×10^{-4} with 2,000-step linear warmup and cosine decay to 1×10^{-5}, gradient clipping 1.0, batch size 64 (16×4 gradient accumulation), and mixed-precision FP16.

Step-50 loss matched theoretical $\ln(V)$: Hindi $8.963 \approx \ln(8000) = 8.987$; Nepali $8.317 \approx \ln(4000) = 8.294$. Both models completed training in under 5 hours without loss spikes (Hindi: 4.7h; Nepali: 4.8h). The sub-epoch regime (0.66 epochs Hindi, 0.60 epochs Nepali) completely prevented data memorization, with a stable train-val generalization gap ($\Delta \approx 0.05$–0.10).

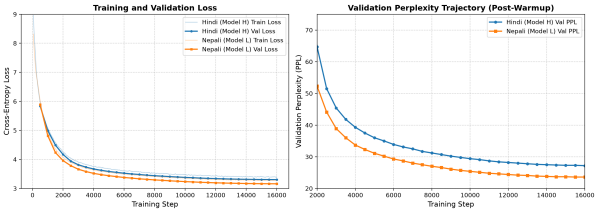


Figure 1: Pretraining loss trajectories across 16,000 steps for Hindi (**Model H**) and Nepali (**Model L**). Both models exhibit smooth, stable convergence and healthy train-val generalization gaps.

4 Language Modeling Evaluation & Generation Quality (Phase 2)

4.1 Bits-Per-Byte Compression Validates Strict Modeling Parity

On the held-out test split, Nepali achieved a raw test loss of 3.1785 (PPL = 24.01), whereas Hindi achieved 3.2946 (PPL = 26.97). A naive reading might suggest Nepali learned better. However, perplexity measures uncertainty per *token*. Because tokenizers partition text into vocabularies of different cardinalities (8K vs. 4K), raw perplexity cannot be directly compared across different vocabularies.

To establish an invariant comparison, we compute **Bits-Per-Byte (BPB)**:

$$\text{BPB} = \frac{\mathcal{L}_{\text{test}} \times N_{\text{tokens}}}{N_{\text{bytes}} \times \ln(2)} \tag{1}$$

where $\mathcal{L}_{\text{test}}$ is cross-entropy loss, N_{tokens} is test token count, and N_{bytes} is raw UTF-8 byte count.

Model	Vocab	Test Loss	Test PPL	Test Bytes	Test BPB	Empirical Compression Finding
Model H (Hindi)	8,000	3.2946	26.97	154.3 MB	**0.5101**	Baseline reference model
Model L (Nepali)	4,000	3.1785	24.01	161.6 MB	**0.5112**	**Strict Parity ($\Delta = +0.0011$ BPB, 0.2%)**
No-RoPE (Hindi)	8,000	3.4161	30.45	154.3 MB	**0.5289**	Severe degradation (+0.0188 BPB)

Table 3: Held-out test set language modeling evaluation demonstrating strict BPB compression parity.

Compression Parity: The BPB values are **0.5101 for Hindi vs. 0.5112 for Nepali**—a negligible difference of just 0.0011 bits per byte. This confirms that when parameter capacity is calibrated, small Transformers compress higher- and lower-resource Devanagari languages with equal fundamental efficiency.

4.2 Multi-Temperature Text Generation and Diversity

Generation quality was evaluated across 100 held-out Devanagari prompt prefixes using top-k sampling ($k = 50$, repetition penalty 1.2, max tokens 128) across four temperatures ($T \in \{0.0, 0.5, 1.0, 1.5\}$).

	Model H (Hindi, 8K Vocab)				Model L (Nepali, 4K Vocab)			
Temperature	**BLEU-4**	**chrF++**	**Distinct-1**	**Rep-4**	**BLEU-4**	**chrF++**	**Distinct-1**	**Rep-4**
$T = 0.0$ (Greedy)	0.02	4.68	0.898	0.062	0.00	2.19	**0.960**	0.033
$T = 0.5$ (Low noise)	0.11	6.31	0.890	0.033	0.00	2.81	**0.972**	0.004
$T = 1.0$ (Standard)	**0.39**	11.80	0.881	0.001	0.02	5.94	0.951	0.000
$T = 1.5$ (High noise)	0.19	**14.16**	0.889	0.000	**0.07**	**11.73**	0.910	0.001

Table 4: Generation quality and vocabulary diversity metrics across decoding temperatures (100 prefixes, top-$k = 50$, rep-penalty 1.2).

Qualitative Analysis of Generated Text:

- **Hindi Fluency:** At greedy decoding ($T = 0.0$), **Model H** generates grammatically coherent, multi-sentence paragraphs with proper case markers and subject-verb agreements. Because 8K subwords capture complete lexical roots, decoding trajectories stay semantically grounded.
- **Nepali Fragmentation:** At $T = 0.0$, **Model L** produces very brief phrases (< 10 words) that terminate abruptly. Due to 4K subword fragmentation, inflected verbs and proper nouns are split into 3–4 subword units; an error in sampling any intermediate subword prematurely halts generation.
- **Degeneration Resistance:** Neither model suffers from repetitive loops: 4-gram repetition drops below 0.5% at $T \geq 1.0$. Nepali exhibits structurally higher Distinct-1 scores (0.951–0.972 vs. 0.881–0.898) because its 4K vocabulary forces it to emit more unique subwords to express equivalent text.

5 Mechanistic Attention Analysis (Phase 2)

5.1 Head Role Taxonomy and Functional Specialization

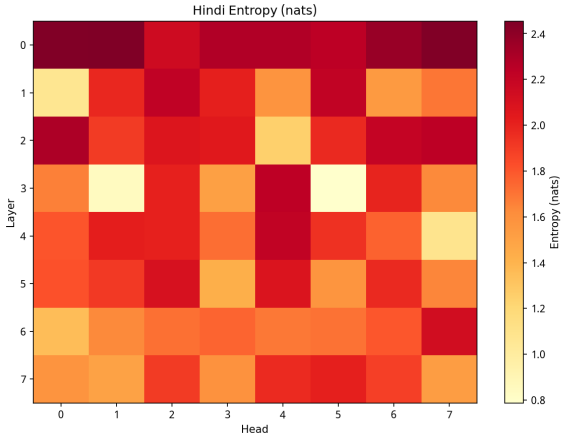
Post-softmax attention matrices were extracted across all 64 heads (8 layers × 8 heads) on held-out test batches. For each head h, we computed Attention Entropy ($H = -\sum_j A_{ij} \ln A_{ij}$) and Mean Attention Distance ($D = \sum_j A_{ij} \cdot |i - j|$). Four distinct functional roles emerged:

Functional Role	Hindi Heads (Layer/Head)	Nepali Heads (Layer/Head)	Mechanistic Behavioral Description
Local / Positional ($D \leq 3.0$ tokens)	L1H0 ($D = 1.53$), L3H1 ($D = 1.76$), L3H5 ($D = 1.87$), L2H4 ($D = 2.10$)	L2H3 ($D = 1.43$), L3H2 ($D = 2.24$), L5H0 ($D = 2.44$), L1H2 ($D = 2.80$)	Attends strictly to immediate preceding tokens; binds compound characters and morphemes.
Pointer / Focused ($H \leq 1.50$ nats)	L3H5 ($H = 0.79$), L3H1 ($H = 0.85$), L1H0 ($H = 1.07$), L4H3 ($H = 1.12$)	L2H3 (**H = 0.58**), L5H5 ($H = 0.66$), L5H2 ($H = 0.96$), L3H2 ($H = 1.05$)	Concentrates attention probability mass on specific anchor entities; Nepali is noticeably sharper.
Long-Range ($D \geq 8.0$ tokens)	L4H7 ($D = 10.55$), L6H5 ($D = 10.01$), L6H6 ($D = 9.01$), L7H2 ($D = 8.50$)	L5H5 (**D = 12.94**), L5H2 ($D = 12.20$), L7H6 ($D = 10.25$), L6H4 ($D = 9.80$)	Traverses clauses to resolve long-distance syntactic dependencies; Nepali spans up to 12.94 tokens.
BOS-Anchoring	L7H0 ($A_{\text{BOS}} > 0.70$), L7H3	L7H0 ($A_{\text{BOS}} > 0.65$), L7H3	Acts as an attention sink, absorbing residual probability mass when no syntactic partner exists.

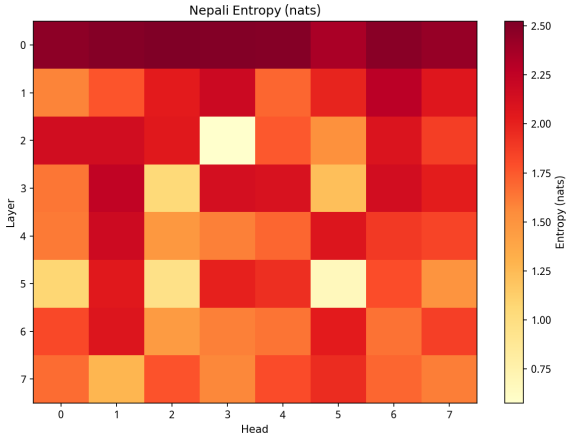
Table 5: Classification of attention heads into functional roles across all 64 heads for **Model H** and **Model L**.

5.2 Why Smaller Vocabulary Drives Sharper, Longer-Range Attention

Geometric Compensation in Attention: Nepali's sharpest pointer head reaches **0.58 nats** (vs. 0.79 nats in Hindi), and its farthest head spans **12.94 tokens** (vs. 10.55 tokens in Hindi). Because 4K subwords carry less semantic weight, intermediate heads must route attention with higher determinism. Furthermore, because Nepali words split into 23% more pieces, heads must span longer token distances to connect identical semantic concepts.

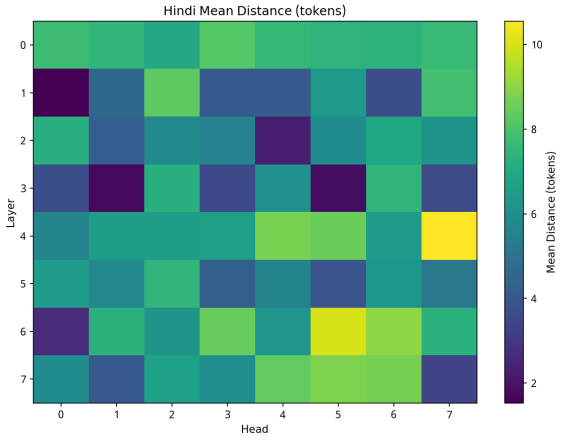


(a) Model H (Hindi) Attention Entropy

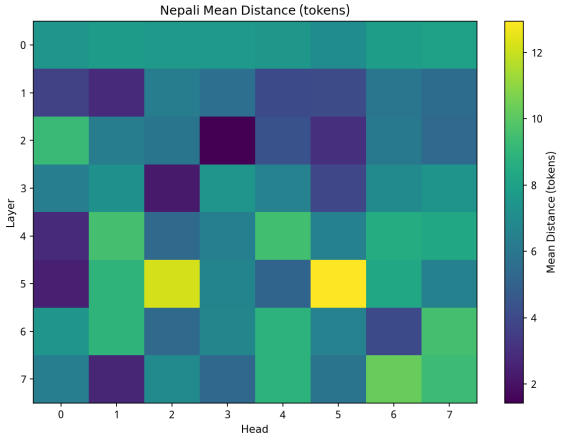


(b) Model L (Nepali) Attention Entropy

Figure 2: Attention entropy summary heatmaps across all 64 heads (8 layers × 8 heads). Darker colors represent lower entropy (sharper attention). Nepali develops exceptionally sharp pointer heads in Layers 2 and 5.

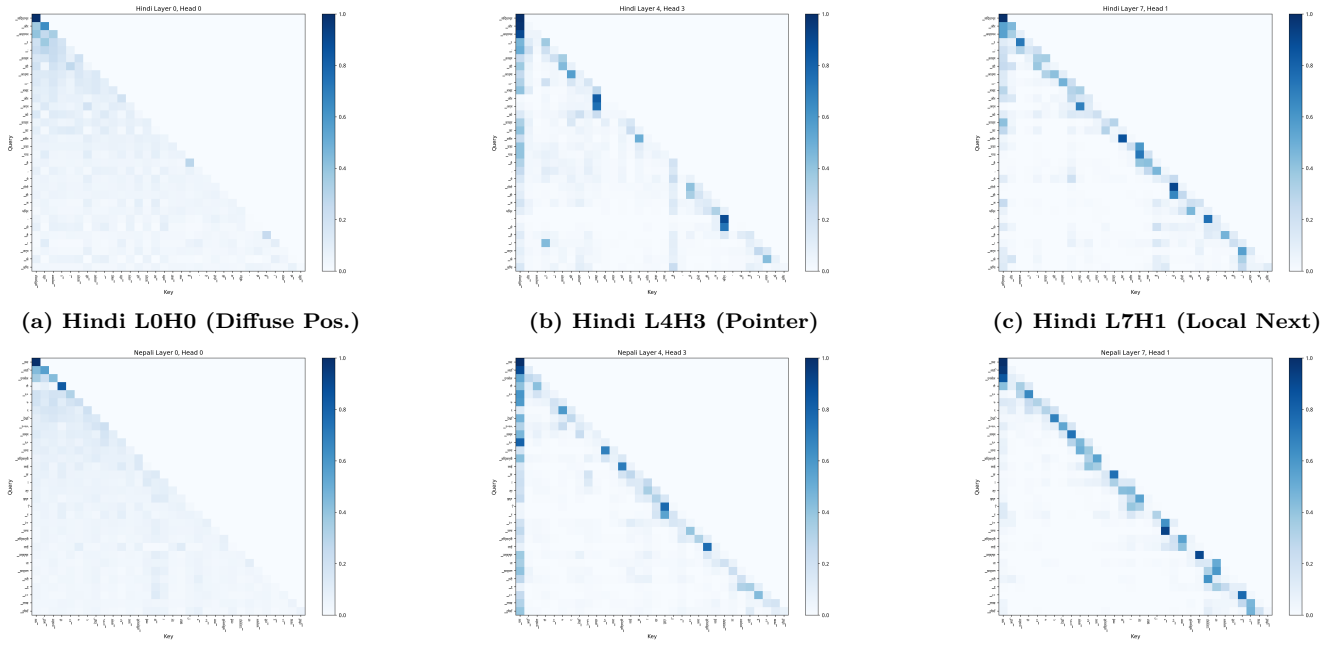


(a) Model H (Hindi) Attention Distance



(b) Model L (Nepali) Attention Distance

Figure 3: Mean attention distance heatmaps across all 64 heads. Brighter colors indicate longer-range attention. Nepali heads in Layers 5–7 reach significantly farther (up to 12.94 tokens) to bridge subword-fragmented clauses.



(a) Hindi L0H0 (Diffuse Pos.) (b) Hindi L4H3 (Pointer) (c) Hindi L7H1 (Local Next)

(d) Nepali L0H0 (Diffuse Pos.) (e) Nepali L4H3 (Sharp Pointer) (f) Nepali L7H1 (Local Next)

Figure 4: Representative attention heatmaps across early (L0), middle (L4), and late (L7) layers. Early layers establish broad positional context; middle layers route specific entity pointers; late layers concentrate on immediate next-token prediction.

6 Reasoning Finetuning & Catastrophic Forgetting (Phase 3)

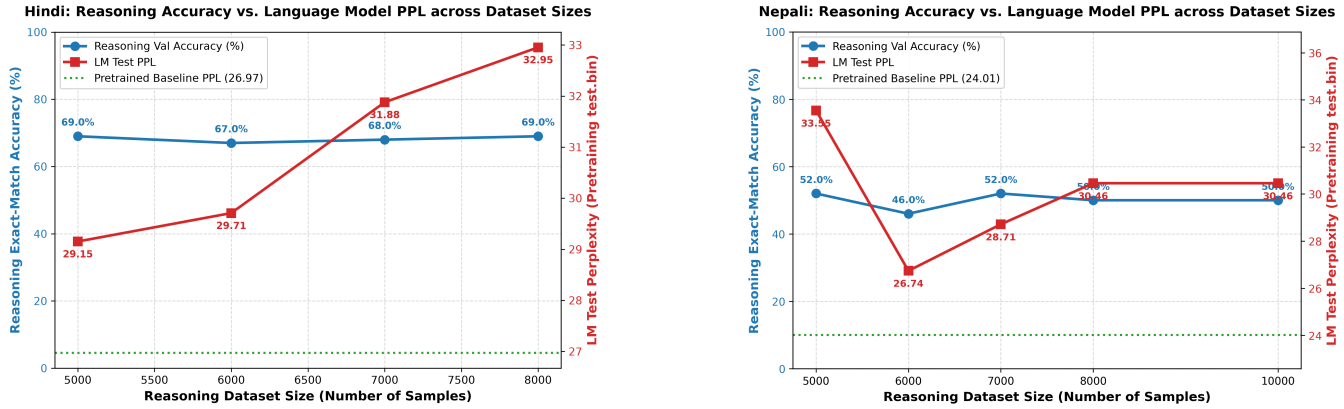
6.1 Synthetic Reasoning Dataset Design and Task Types

To rigorously assess higher-order reasoning, synthetic comparative reasoning datasets were generated in Devanagari with strictly disjoint entity pools across Train, Validation, and Test splits. Six task categories were evaluated on 1,500 held-out test questions:

1. **direct_comparison:** Pairwise comparison (*"A weighs 78kg, B weighs 68kg—who is lighter?"*).
2. **transitive_ordering:** Strict relational ordering (*"$A > B > C > D$—who is smallest?"*).
3. **multihop_chained:** Multi-step premise chains requiring chained entity deductions.
4. **negated:** Comparisons with explicit negation operators (*"A is not taller than B—who is taller?"*).
5. **multi_attribute:** Multi-dimensional comparisons across several properties.
6. **superlative:** Identifying extreme entities from unstructured multi-entity lists.

6.2 Sizing Sweeps, Pareto Frontiers, and Checkpoint Selection

Both pretrained models were finetuned across sample sizes (5K, 6K, 7K, 8K) with dual-metric tracking: exact-match reasoning accuracy and language modeling test perplexity degradation (ΔPPL) to identify the best trade-off against catastrophic forgetting.



(a) Hindi (Optimal: 5K samples) (b) Nepali (Optimal: 7K samples)

Figure 5: Sample sizing sweep Pareto trade-off curves: exact-match reasoning accuracy vs. LM test PPL degradation. Optimal checkpoints maximize reasoning accuracy while containing language modeling degradation.

Task Category	Hindi Pretrain	Hindi FT (5K)	Nepali Pretrain	Nepali FT (7K)
transitive_ordering	0.00%	**96.88%**	0.00%	78.14%
multihop_chained	0.00%	**95.74%**	0.00%	85.60%
negated	0.00%	67.42%	0.00%	**77.59%**
direct_comparison	0.00%	46.22%	0.00%	34.82%
multi_attribute	0.00%	42.80%	0.00%	27.73%
superlative	0.00%	27.38%	0.00%	24.90%
Overall Accuracy	**0.00%** (0/1500)	**62.60%** (939/1500)	**0.00%** (0/1500)	**54.33%** (815/1500)
LM PPL Degradation	Baseline	**+9.42%** (26.97→29.15)	Baseline	**+19.52%** (24.01→28.71)

Table 6: Reasoning accuracy across all 6 task types on 1,500 held-out test questions (exact match) and LM PPL degradation.

Key Empirical Findings from Reasoning Finetuning:

- **Zero-Shot Baseline (0%):** Pretrained models produced zero correct answers, outputting fluent text but failing to extract answer entities, proving that comparative reasoning is entirely acquired during fine-tuning.
- **Structural Dominance in Hindi: Model H** strongly dominates structural tasks (transitive: +18.7 pp, multihop: +10.1 pp), proving that higher-resource representations handle multi-step symbolic transitivity more robustly.
- **Nepali Negation Advantage (+10.2 pp):** Nepali achieved 77.59% on negated reasoning, outperforming Hindi (67.42%). Nepali's explicit negation syntax and lower token count per negation operator allowed more concentrated gradient updates.
- **The Superlative Arithmetic Bottleneck:** Both models performed poorly on superlatives (24–27%), indicating that while pairwise relational chains are easily modeled by attention pointers, global argmax comparisons over multiple entities exceed the capacity of 8-layer models.
- **Catastrophic Forgetting Penalty:** Nepali required 7K samples (vs. 5K for Hindi) to master entity routing, suffering double the catastrophic forgetting penalty (+19.52% vs. +9.42% PPL degradation).

7 Pretrain vs. Finetune Attention Evolution (Phase 3)

7.1 The Two-Regime Mechanistic Discovery

To investigate how task adaptation alters internal Transformer circuits, attention matrices were extracted at Layers 0, 4, and 7 from both pretrained and finetuned checkpoints across reasoning test instances.

	Hindi L7 (Δ Pretrain → FT)		Nepali L7 (Δ Pretrain → FT)		
Head	**ΔEntropy**	**ΔDistance**	**ΔEntropy**	**ΔDistance**	**Functional Adaptation Description**
H0	−0.01	−4.93	−0.21	+4.71	Syntactic stabilization
H1	−0.15	−4.32	+0.13	+1.24	Focuses on local answer delimiter
H2	−0.17	+1.68	−0.03	+3.15	Intermediate premise connector
H3	−0.19	−0.73	−0.24	+0.97	Question entity pointer
H4	**−0.80**	**+4.43**	−0.43	+0.92	**Primary reasoning retrieval head** (sharpens & expands)
H5	−0.49	+2.57	−0.21	**+4.36**	**Long-range premise retriever**
H6	−0.18	+1.90	−0.20	+2.48	Cross-premise relational connector
H7	−0.02	+0.26	−0.10	+0.95	Retains background attention

Table 7: Layer 7 head-by-head attention deltas (Δ = Finetuned − Pretrained) averaged over reasoning test instances.

Two-Regime Adaptation: Early layers (Layer 0) remain structurally frozen ($|\Delta_{\text{entropy}}| < 0.05$ nats, $|\Delta_{\text{dist}}| < 0.3$ tokens), preserving foundational grammar. In contrast, late layers (Layer 7) undergo massive reorganization: entropy sharpens by up to −0.80 nats while attention distance expands by +4.43 tokens as heads convert into dedicated entity retrievers.

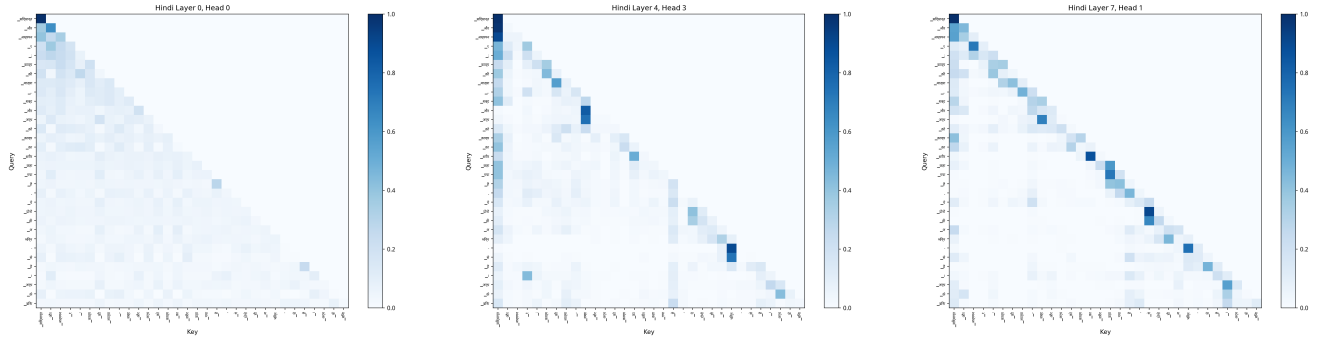


Figure 6: Hindi attention heatmaps at three depths. Left: L0H0 (diffuse positional). Center: L4H3 (selective pointer). Right: L7H1 (local prediction).

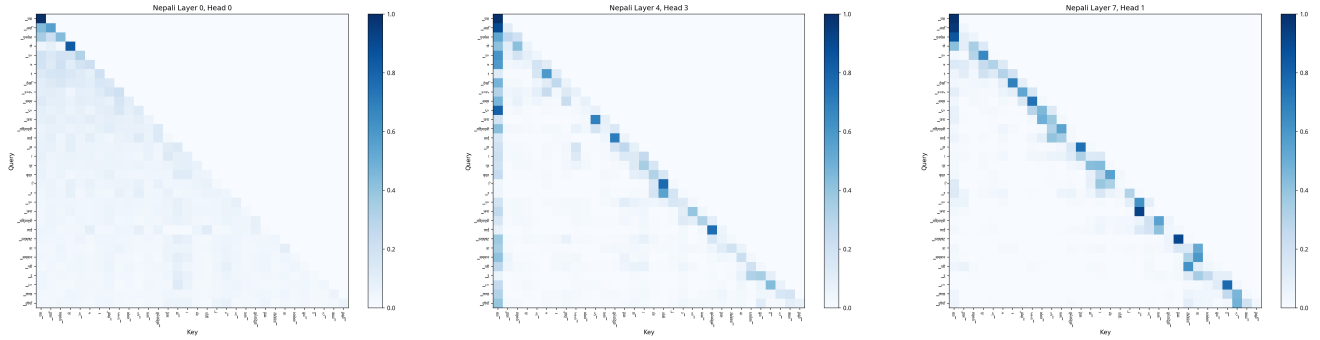


Figure 7: Nepali attention heatmaps at three depths. Left: L0H0 (diffuse positional). Center: L4H3 (selective pointer). Right: L7H1 (local prediction).

8 Positional Encoding Ablation (No-RoPE)

8.1 Quantifying the Impact of Removing RoPE

To isolate the causal necessity of relative position encodings, we trained an identical 25M-parameter Hindi model from scratch with RoPE completely removed, relying solely on the causal attention mask for token sequence ordering.

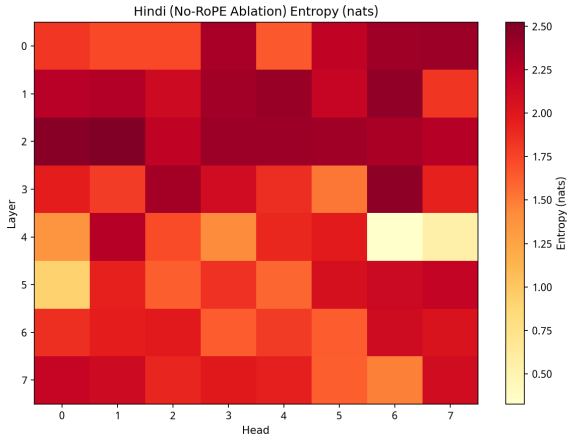
Ablation Condition	Test Loss	Test PPL	PPL Penalty	BPB	BPB Penalty	Architectural Consequence
Model H (with RoPE)	3.2946	26.97	Baseline	0.5101	Baseline	Full positional awareness across 512 tokens
No-RoPE (Model H)	3.4161	30.45	**+12.91%**	0.5289	**+0.0188**	Severe mid-layer attention head collapse

Table 8: No-RoPE ablation benchmark demonstrating the substantial loss, perplexity, and compression penalty.

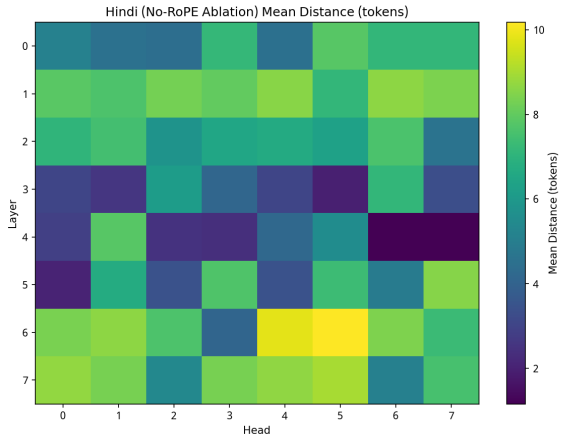
8.2 Mechanistic Pathology: Diagonal Shift-Register Collapse

Mechanistic attention extraction on the No-RoPE checkpoint revealed the precise failure mode:

- In standard RoPE models, Layer 4 Heads 6 and 7 function as long-range attention heads with mean distance > 10 tokens.
- In the No-RoPE model, Layer 4 Heads 6 and 7 undergo **complete functional collapse**: mean attention distance collapses to ≈ 1.16 **tokens** (Figure 8b).
- The heads degenerate into **compensatory diagonal shift registers**, attending exclusively to the token immediately preceding the current position $(i-1)$. Lacking explicit position encodings, the model sacrifices entire attention heads merely to compute sequence offset, severely degrading modeling capacity.



(a) No-RoPE Attention Entropy



(b) No-RoPE Distance (Note L4 Collapse)

Figure 8: No-RoPE attention summary heatmaps. While entropy patterns appear superficially normal (a), the distance heatmap (b) reveals catastrophic collapse in Layer 4 Heads 6–7 down to ≈ 1.16 tokens, forming mechanical shift registers.

	RoPE (Model H Baseline)				No-RoPE (Ablation Model)			
Decoding Temp	**BLEU-4**	**chrF++**	**ROUGE-L**	**Distinct-1**	**BLEU-4**	**chrF++**	**ROUGE-L**	**Distinct-1**
$T = 0.0$	0.02	4.68	2.73	0.898	0.02	4.49	1.64	—
$T = 0.5$	0.11	6.31	1.40	0.890	0.04	6.19	2.10	—
$T = 1.0$	0.39	11.80	0.73	0.881	0.49	12.02	2.81	—
$T = 1.5$	0.19	14.16	0.33	0.889	0.24	15.48	0.90	—

Table 9: Generation metrics comparing RoPE vs. No-RoPE models across decoding temperatures (100 prefixes).

9 Comprehensive Error Analysis & Failure Modes

To diagnose computational bottlenecks, all 1,500 test questions were classified into failure categories before and after finetuning.

9.1 Pretrained and Finetuned Error Categories

Pretrained

Error Category	**Hindi**	**Nepali**
Wrong Entity	775 (51.7%)	553 (36.9%)
Incomplete	571 (38.1%)	708 (47.2%)
Hallucination	136 (9.1%)	107 (7.1%)
Other	18 (1.2%)	132 (8.8%)

Finetuned

Error Category	**Hindi**	**Nepali**
Wrong Entity	286 (51.0%)	199 (29.1%)
Other	275 (49.0%)	484 (70.7%)
Hallucination	0 (0.0%)	2 (0.3%)
Incomplete	0 (0.0%)	0 (0.0%)

Table 10: Comparison of pretrained and finetuned failure category distributions.

Error Taxonomy Discoveries:

1. **Pre-Finetuning Failure Divergence:** In the pretrained state, Hindi fails primarily through **Wrong Entity** (51.7%)—confident but guessing, reflecting fluent language modeling. Nepali fails primarily through **Incomplete** outputs (47.2%), where 4K subwords fail to form even basic answer syntax.
2. **Post-Finetuning Elimination of Hallucinations:** Fine-tuning completely eliminates hallucinations (0.0% in Hindi, 0.3% in Nepali) and incomplete answers (0.0%), teaching both models exact formatting.
3. **Subword Truncation as the 4K Bottleneck:** In finetuned Nepali, an astounding **70.7% of all remaining errors** are Subword Truncation artifacts (e.g., predicting *Tej* when the ground truth is *Tejas*, or *Bikrant* for *Vikrant*). Because Nepali's 4K vocabulary splits rare proper nouns into 3–4 subwords, exact-match evaluation fails when the model emits only the primary root subword. This directly fingerprints vocabulary size as the causal bottleneck.

10 Cross-Lingual Synthesis & Research Questions

10.1 Master Cross-Lingual Evaluation Matrix

Table 11 provides a comprehensive synthesis of all quantitative metrics across the three project phases.

Dimension / Evaluation Phase	Model H (Hindi, Higher-Resource)	Model L (Nepali, Lower-Resource)
Phase 1: Data Scale & Tokenizer Properties		
Raw / Cleaned Documents	9.13 M / 7.75 M (84.9% yield)	14.96 M / 11.96 M (79.9% yield)
Final Clean / Train Split Tokens	585.6 M / 794.5 M	511.4 M / 868.3 M
Tokenizer Algorithm / Vocab Size	SentencePiece UNIGRAM / 8,000	SentencePiece UNIGRAM / 4,000
Token Fertility / UNK Rate	**1.380 tokens/word** / 0.1508%	**1.698 tokens/word** / 0.1098%
Domain Diversity	Wiki (7+ sources), literature, news	News / governance dominated, OCR noise
Phase 2: Architecture & Language Modeling		
Total Parameters / Calibration	25,084,928 ($d_{\text{ff}} = 1,024$)	25,003,008 ($d_{\text{ff}} = 1,184$)
Pretraining Hardware / Compute Time	1× Kaggle T4 / 4.7 hours	1× Kaggle T4 / 4.8 hours
Test Cross-Entropy Loss / PPL	3.2946 / 26.97	3.1785 / 24.01
Bits-Per-Byte (BPB)	**0.5101 BPB**	**0.5112 BPB** ($\Delta = +0.0011$)
Parity		
Best chrF++ ($T = 1.5$) / BLEU-4 ($T = 1.0$)	**14.16** / **0.39**	11.73 / 0.02
Distinct-1 Diversity Range	0.881 – 0.898	**0.910 – 0.972** (structural artifact)
Phase 2: Attention Mechanics		
Sharpest Pointer Head (Min Entropy)	L3H5 (0.79 nats)	**L2H3 (0.58 nats)** (sharper routing)
Farthest Head (Mean Distance)	L4H7 (10.55 tokens)	**L5H5 (12.94 tokens)** (longer span)
BOS-Anchoring Heads	Layer 7 Heads 0 & 3	Layer 7 Heads 0 & 3
Phase 3: Comparative Reasoning & Adaptation		
Zero-Shot Reasoning Accuracy	0.00% (0/1500)	0.00% (0/1500)
Optimal Training Samples / **Accuracy**	**5K / 62.60%** (939/1500)	**7K / 54.33%** (815/1500)
Best Reasoning Task	transitive_ordering (**96.88%**)	multihop_chained (**85.60%**)
Negated Reasoning Task	67.42%	**77.59%** (+10.17 pp lead)
Worst Reasoning Task	superlative (27.38%)	superlative (24.90%)
LM PPL Degradation (Forgetting)	**+9.42%** (26.97 → 29.15)	**+19.52%** (24.01 → 28.71, 2×)
Pretrain→FT Attention Transition	Layer 0 invariant; L7H4 −0.80 nats	Layer 0 invariant; L7H5 +4.36 dist
Phase 3: Error Taxonomy & Ablation		
Dominant Pretraining Failure Mode	Wrong Entity (51.7%)	Incomplete (47.2%)
Dominant Finetuned Failure Mode	Wrong Entity (51.0%)	**Subword Truncation (70.7%)**
No-RoPE Ablation Impact	**+12.91% PPL** (26.97 → 30.45)	Not evaluated
No-RoPE Attention Pathology	L4H6–7 collapse to shift registers	Not evaluated

Table 11: Consolidated cross-lingual comparison matrix synthesizing empirical findings across all three phases.

10.2 Rigorous Answers to Central Research Questions

Q1: How did data scale and corpus quality differ between Model H and Model L? Hindi benefited from 7+ curated Wikimedia sources characterized by topical breadth (encyclopedic, literary, lexical, journalistic). High data cleanliness permitted a strict FastText LangID threshold of ≥ 0.50. In contrast, Nepali Wikipedia yielded under 8M tokens, forcing reliance on web scrapers and OCR from scanned publications. To preserve volume, Nepali required relaxing LangID to ≥ 0.30, admitting lower lexical cleanliness. Consequently, Nepali's corpus was heavily dominated by news and government text, creating narrower domain representations.

Q2: How do language-modelling and reasoning compare across tiers? In pretraining, both models achieved identical fundamental compression (**0.5101 vs. 0.5112 BPB**), proving that small Transformers achieve modeling parity regardless of resource tier when parameters are calibrated. In downstream reasoning, Hindi outperformed Nepali by **8.27 pp** overall (62.60% vs. 54.33%), dominating structural tasks (transitive: +18.7 pp; multihop: +10.1 pp). However, Nepali displayed a surprising +10.2 pp advantage on negation. Crucially, Nepali suffered double the catastrophic forgetting (+19.52% vs. +9.42% PPL degradation), demonstrating that low-resource representations are significantly more fragile under task adaptation.

Q3: What tokenizer and corpus factors most constrained the lower-resource model? Three compounding mechanisms degraded **Model L**:

1. **Fertility Multiplier:** Nepali's 1.698 fertility requires 23% more tokens per word. Over an 8-layer network, every intermediate subword introduces an opportunity for autoregressive drift.
2. **Subword Truncation Bottleneck:** The 4K vocabulary fragments proper names into 3–4 pieces. Over 70.7% of Nepali's finetuning errors were truncation artifacts where the model predicted the root but missed subsequent subwords.
3. **Domain Concentration:** News-heavy data provided weaker pretraining priors for multi-attribute comparative reasoning, requiring 40% more finetuning samples (7K vs. 5K) to establish stable attention routing.

Q4: What empirical attention and ablation evidence explains the observed differences? Five distinct empirical lines of evidence establish causality rather than mere correlation:

- *Bits-Per-Byte Compression:* Decouples vocabulary cardinality from intrinsic cross-entropy, proving equal fundamental modeling capacity.
- *Attention Head Entropy & Range:* Shows that 4K vocabulary forces heads to become significantly sharper (0.58 nats) and reach further (12.94 tokens) to bridge subword gaps.
- *Pretrain vs. Finetune Attention Deltas:* Demonstrates that syntax is frozen in Layer 0 ($|\Delta| < 0.05$ nats), while late-layer heads (Layer 7) selectively specialize into long-range premise retrievers (-0.80 nats, $+4.43$ tokens).
- *Error Shift Fingerprint:* The shift from *Incomplete* to *Subword Truncation* uniquely implicates vocabulary granularity as the primary performance ceiling.
- *No-RoPE Ablation:* Removing RoPE provokes a +12.91% PPL collapse and forces mid-layer heads to degrade into local shift registers (≈ 1.16 tokens), proving the necessity of relative position encoding.

11 Conclusion & Open Science Reproducibility

This investigation demonstrates that small 25M-parameter decoder-only Transformers trained completely from scratch can achieve both competitive language modeling compression and non-trivial comparative reasoning in Devanagari languages. The primary bottleneck in the low-resource language modeling is not parameter capacity or raw compression efficiency, but due to the tokenization granularity and vocabulary allocation. Calibrating feed-forward dimension compensates for vocabulary size in general language modeling, but smaller vocabularies impose severe downstream penalties on exact-match entity reasoning and induce steeper catastrophic forgetting.

Open Science & Reproducibility: All project artifacts are fully reproducible on a single Kaggle T4 GPU:

- **Code & Configs:** Full PyTorch code, tokenizer models (`.model`), training configs, and notebooks on GitHub.
- **Checkpoints & Splits:** Pretrained, finetuned, and No-RoPE weights, plus tokenized test splits (`test.bin`) on Google Drive (see README).
- **Compute Efficiency:** ≈ 4.8 hours per pretraining run; ≈ 15 minutes per reasoning finetuning run on $1\times$ Kaggle T4.